

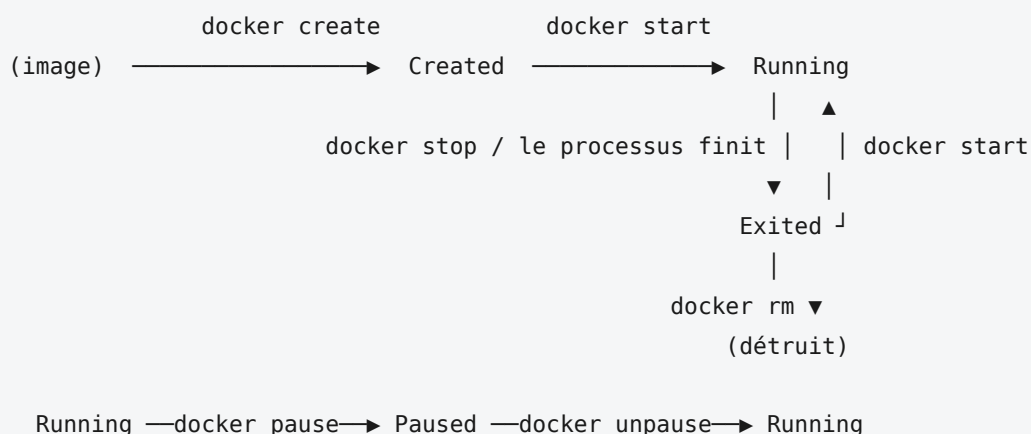
Labo 03 — Cycle de vie d'un conteneur

Théorie — naissance, vie, signaux et mort d'un conteneur, et pourquoi le PID 1 change tout.

Objectifs

- Connaître les états d'un conteneur et les commandes qui font passer de l'un à l'autre.
- Comprendre pourquoi un conteneur « s'arrête tout seul ».
- Maîtriser la relation entre le **processus principal**, les **signaux** et le **code de sortie**.
- Choisir entre `exec` et `attach`, entre premier plan et arrière-plan.
- Savoir ce que fait vraiment une *restart policy*.

1. Les états



Commande	Effet
<code>docker create</code>	Prépare le conteneur (couche d'écriture, config) sans le lancer
<code>docker start</code>	Démarre le processus principal
<code>docker run</code>	<code>create</code> + <code>start</code> (+ <code>pull</code> si l'image manque)
<code>docker stop</code>	Demande poliment l'arrêt, puis force après un délai
<code>docker kill</code>	Force l'arrêt immédiatement
<code>docker restart</code>	<code>stop</code> puis <code>start</code>
<code>docker pause</code>	Gèle les processus (cgroup freezer), sans les arrêter
<code>docker rm</code>	Détruit le conteneur et sa couche d'écriture

Un conteneur `Exited` n'est pas mort : il conserve sa configuration, sa couche d'écriture, ses logs. On peut l'inspecter, le redémarrer, en extraire des fichiers. Seul `docker rm` détruit.

2. La règle fondamentale : un conteneur vit le temps de son PID 1

À retenir — Un conteneur s'arrête exactement quand son **processus principal** se termine. Ni avant, ni après. Il n'y a rien d'autre à comprendre.

C'est l'explication de la quasi-totalité des « mon conteneur s'arrête tout seul » :

- `docker run alpine` sort immédiatement — la commande par défaut est `/bin/sh`, qui, sans terminal attaché, ne lit rien et se termine.
- `docker run nginx` reste en vie — nginx tourne au premier plan et ne rend jamais la main.
- Un conteneur lancé sur un script qui met un service en **arrière-plan** (`java -jar ... &`) meurt aussitôt : le script se termine, donc le PID 1 se termine.

D'où une règle de conception : **une image doit lancer son service au premier plan**. On ne « démontise » pas dans un conteneur, on ne lance pas `systemd`, on n'utilise pas `nohup`. C'est Docker qui joue le rôle du gestionnaire de services.

Corollaire : un conteneur est fait pour **un processus principal**. Mettre l'API et la base dans le même conteneur casse le modèle — on ne peut plus les redémarrer, les surveiller ni les mettre à l'échelle séparément.

3. Premier plan, arrière-plan, et le duo `-it`

```
docker run nginx           # premier plan : le terminal est bloqué, logs affichés
docker run -d nginx        # détaché : rend la main, affiche l'ID du conteneur
docker run -it alpine sh    # interactif : on obtient un shell utilisable
docker run --rm alpine date # exécution ponctuelle, conteneur supprimé à la sortie
```

Le duo `-it` est mal compris parce que ce sont **deux options distinctes** :

- `-i` (`--interactive`) garde l'entrée standard ouverte : sans elle, le processus voit son `stdin` fermé et un shell se termine aussitôt.
- `-t` (`--tty`) alloue un pseudo-terminal : c'est lui qui apporte le prompt, l'écho des touches, la coloration, la gestion de `Ctrl+C`.

En script ou en CI, on utilise `-i` seul (pas de terminal disponible) ; en usage humain, `-it`. Un `-t` seul donne un affichage correct mais un shell qui n'obéit pas au clavier.

Piège — En premier plan, `Ctrl+C` envoie `SIGINT` au processus du conteneur : nginx s'arrête. En mode `-it`, la séquence `Ctrl+P` puis `Ctrl+Q` permet au contraire de se **détacher sans arrêter** le conteneur.

4. Signaux : comment un conteneur meurt

`docker stop` n'est pas un interrupteur. Il déroule un protocole :

1. Envoi de `SIGTERM` au PID 1 : « termine-toi proprement ».
2. Attente d'un **délai de grâce**, 10 secondes par défaut (`-t` pour le changer).
3. Si le processus est toujours là, envoi de `SIGKILL`, non interceptable, immédiat.

`docker kill` saute directement à l'étape 3.

Cette distinction est capitale en entreprise. Pendant les 10 secondes, une application Spring Boot bien écrite termine les requêtes en cours, ferme le pool de connexions PostgreSQL, se désinscrit du service de découverte (*graceful shutdown*, activé par `server.shutdown=graceful`). Avec `SIGKILL`, rien de tout cela : requêtes coupées, connexions laissées ouvertes côté base, éventuellement des données incohérentes.

Deux pièges qui empêchent la réception du signal :

1. Un shell intercalé devant l'application. C'est le cas d'un script de démarrage qui lance l'application sans `exec`, ou de la forme `shell` d'un `CMD` (`CMD java -jar app.jar` devient `/bin/sh -c "java -jar app.jar"`). Le PID 1 est alors `sh`, qui **ne transmet pas** `SIGTERM` à son enfant : Java ne reçoit jamais le signal, attend 10 secondes, puis est tué brutalement. La parade est double : forme `exec` (`CMD ["java", "-jar", "app.jar"]`) et, dans un script, `exec java -jar app.jar` en dernière ligne. Ce détail de syntaxe, détaillé au labo 04, décide de la qualité de vos arrêts.

2. Le statut particulier du PID 1. Le noyau Linux traite le PID 1 à part : il ignore les signaux pour lesquels aucun gestionnaire n'a été installé. Un processus qui ne gère pas explicitement `SIGTERM` et qui tourne en PID 1 est donc **insensible** à `docker stop`, puis tué au bout du délai. C'est aussi le PID 1 qui doit « adopter » les processus orphelins ; un PID 1 qui ne le fait pas laisse s'accumuler des *zombies*. D'où l'option `--init`, qui insère un mini-init correct devant votre application.

5. Codes de sortie

```
docker run --rm alpine sh -c 'exit 3'; echo $?      # 3
docker ps -a --format 'table {{.Names}}\t{{.Status}}'
```

Le code de sortie du conteneur est celui de son PID 1, et il est conservé dans son statut (`Exited (3)`). Quelques codes à reconnaître :

Code	Signification usuelle
0	Terminaison normale
1	Erreur applicative générique
125	Le daemon Docker lui-même a échoué (option invalide)
126	La commande a été trouvée mais n'est pas exécutable
127	Commande introuvable dans l'image
137	Tué par <code>SIGKILL</code> (128+9) — <code>docker kill</code> , ou l' OOM killer
143	Terminé par <code>SIGTERM</code> (128+15) — un <code>docker stop</code> propre

137 est le code que vous verrez le plus en production : soit un `stop` qui a dépassé le délai de grâce, soit un dépassement de la limite mémoire. `docker inspect` tranche : `.State.OOMKilled` vaut `true` dans le second cas.

6. Les restart policies

```
docker run -d --restart=unless-stopped --name api mon-api:1.0
```

Politique	Comportement
<code>no</code> (défaut)	Aucun redémarrage automatique
<code>on-failure[:N]</code>	Redémarre si le code de sortie est $\neq$ 0, au plus N fois
<code>always</code>	Redémarre toujours, y compris après un reboot de l'hôte
<code>unless-stopped</code>	Comme <code>always</code> , sauf si vous l'avez arrêté manuellement

Docker applique un délai croissant entre les tentatives (100 ms, 200 ms, 400 ms...) pour ne pas saturer la machine avec un conteneur qui échoue en boucle.

Piège — `always` redémarre un conteneur même après un `docker stop` ... au prochain démarrage du daemon. `unless-stopped` mémorise votre intention et est presque toujours le bon choix pour un service sur une machine unique.

7. Observer et intervenir

```

docker logs -f --tail 50 api      # flux de sortie du PID 1
docker exec -it api sh            # nouveau processus DANS le conteneur
docker attach api                 # se rebrancher sur le PID 1 existant
docker top api                    # processus du conteneur, vus de l'hôte
docker stats api                  # consommation CPU/mémoire en temps réel
docker inspect api                # état complet, JSON
docker cp api:/app/log.txt .      # extraire un fichier, même conteneur arrêté

```

`exec` et `attach` sont souvent confondus : `exec` **crée un nouveau processus** dans les namespaces du conteneur — c'est ce que vous voulez pour aller voir ce qui s'y passe. `attach` vous rebranche sur l'entrée/sortie du **PID 1 existant** : un `Ctrl+C` y arrête donc le conteneur.

`docker logs` ne montre que ce que le PID 1 a écrit sur `stdout` / `stderr`. Une application qui écrit dans un fichier (`/var/log/app.log`) n'apparaîtra pas — d'où la règle des applications conteneurisées : **logger sur la sortie standard**. Pour Spring Boot, c'est le comportement par défaut ; il ne faut donc surtout pas configurer un `logging.file.name`.

8. En entreprise

- Le conteneur du back Spring Boot tourne en `-d`, avec `--restart=unless-stopped` sur une machine unique — ou sans politique du tout sous un orchestrateur, qui gère lui-même les redémarrages.
- L'arrêt propre est un sujet de production : `SIGTERM` correctement reçu + `graceful shutdown` Spring = déploiements sans requête perdue.
- Le diagnostic d'un incident suit toujours le même enchaînement : `docker ps -a` (quel statut, quel code), `docker logs` (qu'a dit l'application), `docker inspect` (OOM ? quelle configuration ?), puis `docker exec` si le conteneur vit encore.

À retenir

- Un conteneur vit exactement le temps de son processus principal (PID 1).

- Il faut lancer les services **au premier plan** : ni démon, ni `&`, ni `systemd`.
- `-i` garde `stdin` ouvert, `-t` alloue un terminal ; ce sont deux choses différentes.
- `docker stop` = `SIGTERM`, délai de grâce, puis `SIGKILL`. `docker kill` = `SIGKILL` direct.
- La forme `exec ( ["java", "-jar", "x.jar"] )` est indispensable pour recevoir les signaux.
- `137` = tué (KILL ou OOM), `143` = arrêté proprement (TERM), `127` = commande introuvable.
- `docker rm` détruit les données du conteneur ; `docker stop` non.
- `exec` crée un processus, `attach` se branche sur le PID 1 existant.

Vocabulaire

PID 1 : processus principal du conteneur. — **détaché** (`-d`) : lancé en arrière-plan. — **TTY** : pseudo-terminal, apporté par `-t`. — **délai de grâce** : temps laissé entre `SIGTERM` et `SIGKILL`. — **graceful shutdown** : arrêt propre d'une application qui termine son travail en cours. — **restart policy** : règle de redémarrage automatique. — **OOM killer** : mécanisme du noyau qui tue un processus quand la mémoire manque. — **zombie** : processus terminé dont personne n'a lu le code de sortie.